

Project: Project Assignment and Scheduling

Abstract

The Project Assignment and Scheduling (PA&S) problem involves assigning tasks to teams while respecting precedence and availability constraints, aiming to maximize task completion, minimize time and cost. This study aims to highlight three models's strengths and limitations: a Constraint Programming (CP-SAT) model, a greedy model, and a backtracking model.

Overall, CP-SAT provides the best solution quality when time and problem size permit. Greedy is fast and scales well but may miss optimal results. Backtracking finds optimal solutions if given time, but is impractical for large inputs without time limits or pruning.

Description

A project consists of N tasks $1, 2, \dots, N$ need to be completed by M teams $1, 2, \dots, M$.

Team j is available at time point $s(j)$.

A task i has a duration $d(i)$ and can be performed by some teams: $c(i,j)$ is the cost if team j performs the task i .

Between N tasks, there are precedence constraints represented by Q pairs (i,j) in which task j can only be started to execute after the completion of task i .

Compute the schedule for performing N tasks (assign a team for each task and specify the starting time-point for each that task) optimizing following criteria (prioritize in the given order):

1. Number of tasks are scheduled is maximal
2. The completion time of all tasks is minimal
3. The total cost for the task assignment is minimal

Input

- Line 1: contains 2 integers N and Q ($1 \leq N, Q \leq 1000$)
- Line $i+1$ ($i = 1, 2, \dots, Q$): contains i and j in which task j can only be started to execute after the completion of task i
- Line $Q+2$: contains N positive integers $d(1), d(2), \dots, d(N)$ ($1 \leq d(i) \leq 1000$)
- Line $Q+3$: contains a positive integer M ($1 \leq M \leq 500$)
- Line $Q+4$: contains M positive integers $s(1), s(2), \dots, s(M)$ ($1 \leq s(i) \leq 1000$)
- Line $Q+5$: contains a positive integer K ($1 \leq K \leq 1000000$)
- Line $Q+5+k$ ($k = 1, 2, \dots, K$): contains i, j, and $c(i,j)$ in which $c(i,j)$ is the cost when assigning team j to task i

Output

- Line 1: contains a positive integer R
- Line $i + 1$ ($i = 1, \dots, R$): contains 3 positive integer i, j, and u in which task i is assigned to team j and is started to execute at time-point u

Model analysis

CP-SAT

a. Processing input

```
lines = []
try:
    while True:
        line = input()
        if line.strip() == "":
            break
        lines.append(line)
except EOFError:
    pass
```

This collects multiple lines of input and stores them in a list called **lines** until the user enters a blank line or triggers an end-of-file (EOF).

We parse the number of tasks (n) and constraints (q) by:

```
idx = 0
n, q = map(int, lines[idx].split()); idx += 1
```

To read the constraints: task u must finish before task v starts:

```
pre = [tuple(map(int, lines[idx + i].split())) for i in range(q)]; idx
+= q
```

To read the durations of each task:

```
dur = list(map(int, lines[idx].split())); idx += 1
```

To read the number of teams:

```
m = int(lines[idx]); idx += 1
```

To read the availability time of each team:

```
avail = list(map(int, lines[idx].split())); idx += 1
```

We choose to read and store the task-team-cost relationships by a dictionary:

```
k = int(lines[idx]); idx += 1
cost = defaultdict(lambda: float('inf'))
```

The default cost is set to infinity by default, indicating impossibility if assigning a task to a team that cannot do that task.

```
can_do = [[] for _ in range(n)]
```

This initialize lists inside a list of available teams to each tasks.

```
for i in range(k):
    t, tm, c = map(int, lines[idx + i].split())
    t -= 1
    tm -= 1
    cost[(t, tm)] = c
    can_do[t].append(tm)
```

Beside updating the dictionary cost and appending to the list can_do, this loop also returns the task and team values to 0-indexing for better uses of programs in the main model.

As output on HUSTack require printing values in the 1-indexing format, we will later add 1 to each value before printing.

We can detect tasks that no team can do (blocked) to reduce workload when optimizing:

```
blocked = set()
for i in range(n):
    if not can_do[i]:
        blocked.add(i)
```

All tasks depending on blocked ones are also blocked:

```
queue = deque(blocked)
while queue:
    i = queue.popleft()
    for u, v in pre:
        if u - 1 == i and v - 1 not in blocked:
            blocked.add(v - 1)
            queue.append(v - 1)
```

b. Declaring values

Declare an upper bound for calculation by:

```
hz = sum(dur) + max(avail)
```

This assumes the worst case that no task can be executed simultaneously/ all tasks are done by 1 team, and that team has the latest available time.

Declaring an upper bound is crucial because CP-SAT model needs a finite domains of variables. This also allows a faster computing time and prevent time-outs for large inputs.

Next, we declare decision variables based on the output requirements on HUSTack:

- $s[i]$ - start time of task i :

```
s = [None if i in blocked else model.NewIntVar(0, hz, f's_{i}') for i in range(n)]
```

- $p[i]$ - whether task i is scheduled:

```
p = [None if i in blocked else model.NewBoolVar(f'p_{i}') for i in range(n)]
```

- $x[(i,j)]$ – whether task i is assigned to team j :

```
x = {}
intervals = [[] for _ in range(m)]
for i in range(n):
    if i in blocked:
```

```

        continue
    for j in can_do[i]:
        x[(i, j)] = model.NewBoolVar(f'x_{i}_{j}')

```

c. Constraints

- If task i is scheduled, it must be assigned to exactly one team that can do it:

```

for i in range(n):
    if i in blocked:
        continue
    model.Add(sum(x[(i, j)] for j in can_do[i]) == p[i])

```

- If task i is assigned to team j, the starting time of task i must be bigger than the available time of team j:

```

for j in can_do[i]:
    model.Add(s[i] >= avail[j]).OnlyEnforceIf(x[(i, j)])

```

- If both u and v are scheduled and task u is the precedent of task v, then task v can only start after task u ends:

```

for u, v in pre:
    u -= 1
    v -= 1
    if u in blocked or v in blocked:
        continue
    model.Add(s[v] >= s[u] + dur[u]).OnlyEnforceIf([p[u], p[v]])

```

We also return the values to 0-indexing before declaring variables.

- No two tasks assigned to the same team may overlap:

There is an interesting function in CP-SAT that allows tracking each team's intervals:

```

for i in range(n):
    if i in blocked:
        continue
    for j in can_do[i]:
        iv = model.NewOptionalIntervalVar(s[i], dur[i], s[i] + dur[i],
x[(i, j)], f'i_{i}_{j}')
        intervals[j].append(iv)

```

Optional interval variable is a powerful concept in constraint programming with OR-Tools, especially for scheduling problems. Inside the bracket is the form (start, duration, end, only active if a boolean variable is true). We need optional interval variables instead of fixed, because not every task is assigned, and not every team can do every task, so we only activate an interval for team j if it is actually assigned task i, allowing solver to explore many possible assignments.

We store those intervals per team and finally add the constraint by:

```
intervals[j].append(iv)
for j in range(m):
    model.AddNoOverlap(intervals[j])
```

We then calculate makespan by the latest scheduled task end time, guarantee by the constraint:

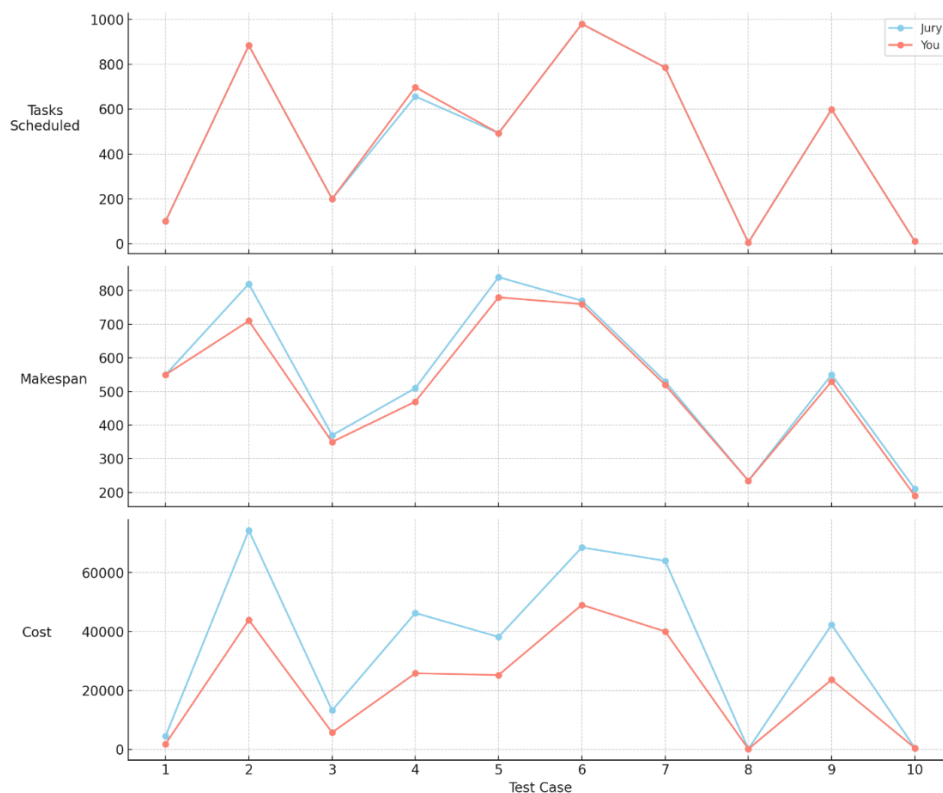
```
ms = model.NewIntVar(0, hz, 'ms')
for i in range(n):
    if i in blocked:
        continue
    model.Add(ms >= s[i] + dur[i]).OnlyEnforceIf(p[i])
```

The core of this optimization program lies in:

```
model.Maximize(sum(p[i] for i in range(n) if i not in blocked) *
1000000 - ms * 1000 - tc)
```

This function allows optimizing in desired order: maximize number of scheduled tasks – minimize makespan – then minimize cost, by multiplying biggest weight for the most prioritized task.

Result



CP-SAT method is undeniably one of the best approaches for this problem, as the program's output closely match the jury's in terms of task completed – the most prioritized criteria, with slight improvement in test case 4. The makespan of the program's output is consistently lower, with noticeable difference in test case 2, while the cost is usually just equal to half of the jury's.

One downside is that this method took extremely long runtime due to the model's nature.

Greedy

a. Processing input:

I used the same style as CP-SAT, with minor changes of creating a reverse dependency list so `pre_map[v]` gives all prerequisites of task `v`:

```
pre_map = [[] for _ in range(n)]
for u, v in pre:
    pre_map[v].append(u)
```

b. Greedy assignment

Initialization:

```
done = set() # scheduled tasks
done_time = [0] * n # end time of task i
res = {} # final schedule results
```

The next loop lasts until all non-blocked tasks are scheduled:

```
while len(done) < n - len(blocked):
    ready = [i for i in range(n) if i not in done and i not in blocked
             and all(u in done for u in pre_map[i])]
    if not ready:
        break
```

Tasks are ready if not blocked, not done, and all their prerequisites are done.

```
if not ready:
    break
```

We exit loop if there are no schedulable tasks remaining.

```
ready.sort(key=lambda i: -dur[i])
```

This function sorts ready tasks by decreasing duration, prioritizing longer tasks.

```

for i in ready:
    teams = can_do[i]
    if not teams:
        continue

```

Then iterates over ready tasks and find the teams that can do it.

```

prereq_done = max((done_time[u] for u in pre_map[i]), default=0)

```

This function finds the latest time all prerequisite tasks of task i finished.

```

j = min(teams, key=lambda t: (max(avail[t], prereq_done), cost[(i,
t)]))

```

Then we chooses the best team that can start the earliest, breaking ties by its cost.

```

st = max(avail[j], prereq_done)
et = st + dur[i]

```

We also computes start and end times of task i.

```

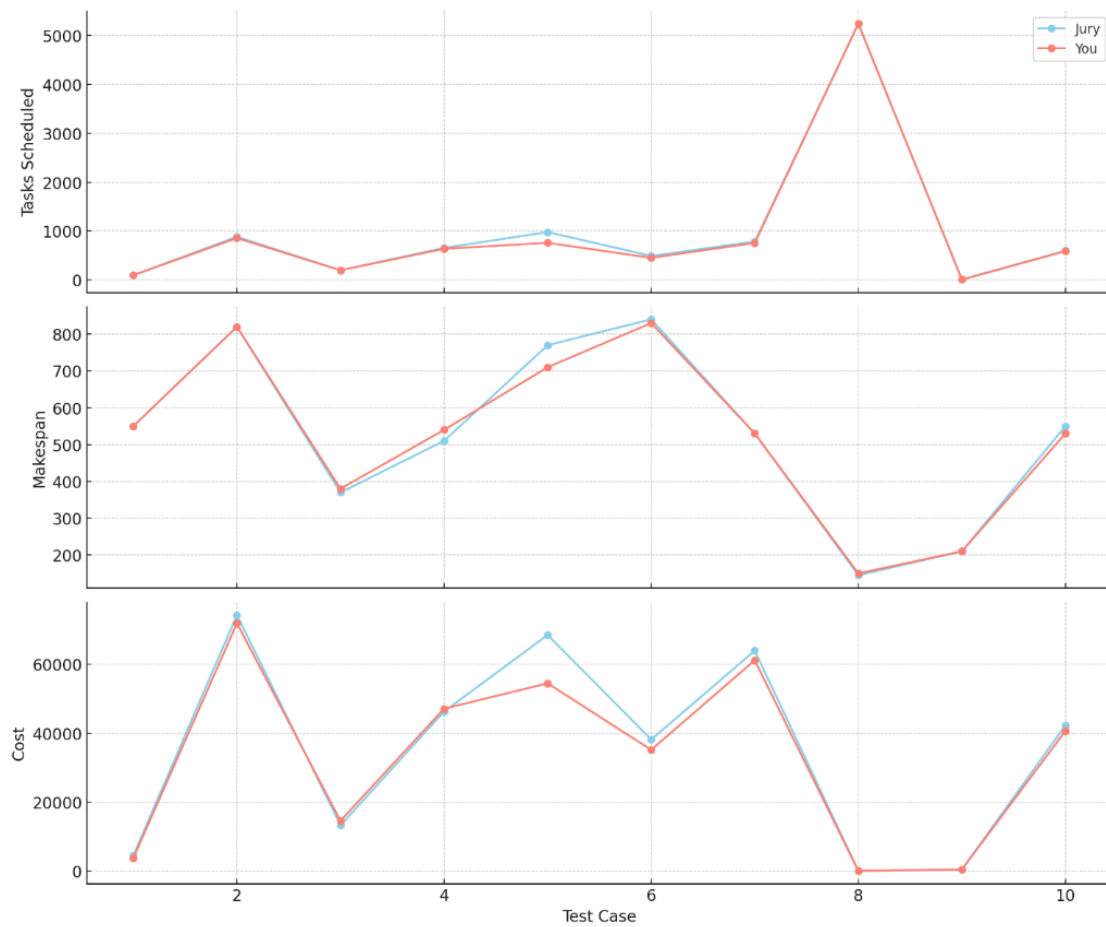
res[i] = (j, st)
avail[j] = et
done_time[i] = et
done.add(i)

```

After scheduling the task i, this function saves it in result (res), then updates team j availability, and marks task i as done by appending it to the list done.

When this loop finished, it will print out the result in 1-index format as required.

Result



With a remarkable running time of just more than 1 second for 10 test cases (complexity of $O(N*M)$, scale linearly, no recursion), with most of them having > 500 lines of input, and produce a fairly high score, greedy became the ideal approach for very large test cases that needs a fast runtime.

Backtracking

All input processing are the same as 2 above approaches. The core of this method lies in the function `backtrack()`

```
def backtrack(idx, cnt, ms, total_cost):
```

This defines the recursive backtracking function with 4 state-tracking parameters:

- `idx`: index in the order list (i.e., which task to consider next)
- `cnt`: number of tasks scheduled so far

- ms: current makespan (latest finish time of any task)
- total_cost: cumulative cost of the current schedule

```

if idx == len(order):
    score = (cnt, -ms, -total_cost)
    if score > bscore:
        best[:] = list(sched)
        bscore = score
    return

```

If all tasks in order have been visited (idx == len(order)):

- A complete scheduling decision has been reached.
- Compute the score tuple.
- If this score is better than the current best:
 - Update best to be a copy of sched
 - Update bscore

```
i = order[idx]
```

This retrieves the next task i from the topological order to consider.

```

for j in sorted(can_do[i], key=lambda x: (cost[(i, x)],
avail[x])):

```

Iterates over all valid teams j that can perform task i.

Teams are sorted by:

- Lower cost cost[(i, j)]
- Earlier availability avail[j]

```

st = max(pre_end, avail[j])
et = st + dur[i]

```

st: earliest time the team can start the task, considering:

- All prerequisites completed (pre_end)
- Team is available (avail[j])

et: corresponding end time of the task

```
if all(et <= s or st >= e for s, e in busy[j]):
```

This verifies that team j is free in the interval [st, et) and ensures no overlap with previously scheduled tasks.

```
    busy[j].append((st, et))
    sched.append((i, j, st))
    backtrack(idx + 1, cnt + 1, max(ms, et), total_cost +
cost[(i, j)])
    sched.pop()
    busy[j].pop()
```

The function above temporarily assigns task i to team j, then adds the interval to busy[j], and adds task i to current schedule sched.

Then the recursion starts:

- idx + 1: move to next task
- cnt + 1: one more task is scheduled
- max(ms, et): update makespan
- Add cost for this assignment

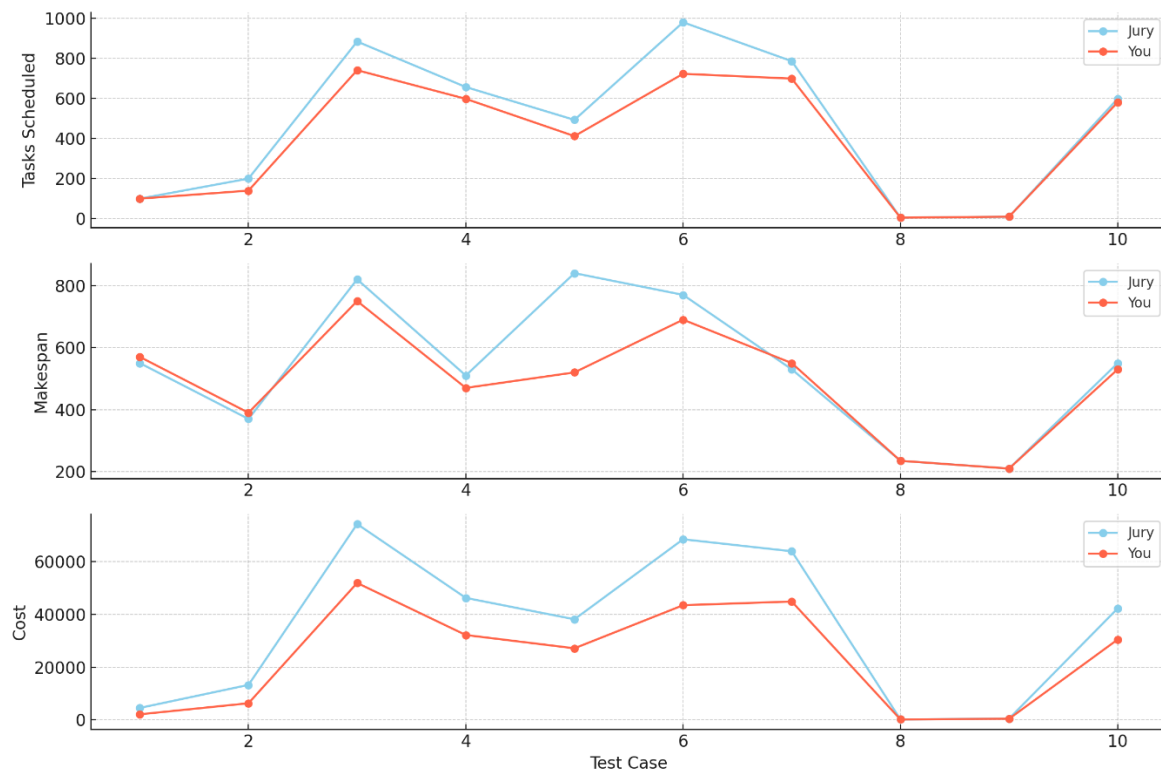
After returning, undo the assignment (backtrack) by popping from both sched and busy[j].

```
backtrack(idx + 1, cnt, ms, total_cost)
```

We also explore the option of skipping task i entirely, by not assigning it to any team, and not increasing the task count, makespan, cost.

This allows incomplete solutions, which are important when some tasks cannot be scheduled.

Result



Because of time limitations, this backtracking approach tends to sacrifice the number of completed tasks for better makespan and cost. The score are still fairly high though (880/1000).

Another downside is that backtracking took an immense amount of computing time due to its complexity of $O(2^n \times M)$ in the worst case:

- For N tasks, each may be scheduled or skipped $\rightarrow 2^N$ combinations
- For each scheduled task, it may try multiple teams (up to M)

It explores all valid combinations, hence exponential upper bound. This method is usable only for small N (~ 20 – 30 tasks) or when bounded by a time limit (in this case is 0.5) or aggressive pruning.

Conclusion

CP-SAT offers the strongest guarantees of optimality when time and problem size allow. It is well-suited for small to medium problem sizes but can slow down significantly as the number of tasks increases due to the complexity of solving large constraint models.

Greedy, in contrast, performs extremely well in terms of runtime and can handle large-scale inputs quickly, although it may miss globally better solutions due to its locally optimal decisions.

Backtracking guarantees optimal results if allowed to fully search the solution space, but its exponential nature makes it impractical for large inputs unless tightly time-limited or enhanced with pruning techniques.

Ultimately, the choice depends on the balance between performance and precision: CP-SAT for guaranteed quality, greedy for speed, and backtracking for exhaustive evaluation under time control.

Future direction

The multi-objective nature of the Project Assignment and Scheduling problem presents both challenges and opportunities. It allows flexible prioritization to fit different needs, but also makes it hard to balance priorities without affecting performances. Future researches should focus on improving time and space efficiency while allowing easier adjustment of constraints and priorities.

Another promising direction is developing hybrid models that combine the strengths of CP, MIP, and Heuristic approaches to better handle large, complex problems.

Finally, while this study focused on theory, future work should also test these models in real-world settings to utilize their practical uses.